

CPSC 250

Inheritance Philosophy in Python

Edward Brash

1 Inheritance, Constructors, and `super()`

1.1 Introduction

In our discussion of inheritance, we learned that one class can inherit data and behavior from another class.

Consider the following example:

```
class Person:
    def init(self, name):
        self.name = name

    def introduce(self):
        print(f"Hello, my name is {self.name}")

class Student(Person):
    pass
```

Since `Student` inherits from `Person`, every `Student` object has a `name` attribute and can use the `introduce()` method.

```
s = Student("Alice")
s.introduce()
```

Output:

Hello, my name is Alice

There is actually something really subtle going on here! Because the sub-class does not have an `__init__` method at all, it actually inherits that method from the base class! In this particular case, it works, *BUT* it is a dangerous game, as we will see below.

1.2 A Problem Appears

Suppose we modify the `Student` class:

```
class Student(Person):
    def init(self, name, student_id):
        self.student_id = student_id
```

Now consider:

```
s = Student("Alice", "C001")
```

The program runs successfully.
However, something important has happened.
The constructor of the **Person** class never executed.
As a result, the object contains:

```
self.student_id
```

but it does not contain:

```
self.name
```

Attempting to use:

```
s.introduce()
```

will produce an error because **name** was never initialized.

1.3 The Solution: Calling the Base Class Constructor

When a derived class defines its own constructor, it becomes responsible for initializing the inherited portion of the object.

Python provides the **super()** function for this purpose.

```
class Student(Person):  
    def init(self, name, student_id):  
        super().init(name)  
        self.student_id = student_id
```

Now the initialization process occurs in two stages:

1. The **Person** constructor initializes the inherited data.
2. The **Student** constructor initializes the new data.

The resulting object contains both attributes:

```
s = Student("Alice", "C001")  
  
print(s.name)  
print(s.student_id)
```

Output:

```
Alice  
C001
```

1.4 Why Doesn't Python Do This Automatically?

Students often ask why Python requires an explicit call to the base-class constructor.

Languages such as Java and C++ automatically invoke the parent constructor.

Python takes a different approach.

One of Python's design principles is:

Explicit is better than implicit.

If a programmer overrides a method, Python assumes that the programmer intends to control what happens next.

Since `__init__()` is simply another method, Python does not automatically call the version defined in the parent class.

Instead, the programmer must explicitly decide whether the parent constructor should execute.

1.5 A Deeper Question: What Is a Constructor?

Interestingly, `__init__()` is not actually responsible for creating an object.

When Python executes:

```
s = Student("Alice", "C001")
```

the following steps occur:

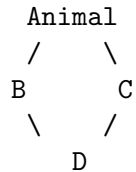
1. Memory is allocated for the new object.
2. The object is created.
3. `__init__()` initializes the object's data.

Thus, `__init__()` is better thought of as an initialization method rather than a true constructor. The object already exists before `__init__()` begins executing.

1.6 The Diamond Problem

The real reason Python's inheritance system is designed this way becomes apparent when multiple inheritance is involved.

Consider the following inheritance diagram:



Notice that class D inherits from both B and C.

Both B and C inherit from Animal.

A natural question arises:

How many times should the `Animal` constructor run?

Once?

Twice?

This ambiguity is known as the **Diamond Problem**.

1.7 Method Resolution Order (MRO)

Python solves this problem by defining a Method Resolution Order (MRO).

The MRO specifies the order in which Python searches for methods.

Consider:

```
class Animal:
    pass

class B(Animal):
    pass

class C(Animal):
    pass

class D(B, C):
    pass

print(D.mro())
```

Output:

```
[D, B, C, Animal, object]
```

This list tells Python exactly how methods should be located.

1.8 The Meaning of `super()`

Many beginning programmers believe that `super()` means:

Call my parent class.

This is not quite correct.

A more accurate description is:

Call the next class in the Method Resolution Order.

For ordinary single inheritance, these two interpretations happen to be the same.

For multiple inheritance, however, the distinction becomes important.

1.9 Writing Constructors Correctly in the Diamond Pattern

Now that we know that `super()` follows the Method Resolution Order, we can rewrite the diamond example correctly.

The key rule is:

Every class in the hierarchy should call `super().init()` exactly once.

Consider the following version:

```
class Animal:
    def init(self):
        print("Animal initialized")

class B(Animal):
    def init(self):
        super().init()
        print("B initialized")

class C(Animal):
    def init(self):
        super().init()
        print("C initialized")

class D(B, C):
    def init(self):
        super().init()
        print("D initialized")
```

Now create an object of type D:

```
d = D()
```

Output:

```
Animal initialized
C initialized
B initialized
D initialized
```

This may seem surprising at first. Why does C initialize before B finishes?

The answer is that `super()` does not mean “call my parent class.” It means:

Call the next class in the Method Resolution Order.

For this class hierarchy:

```
print(D.mro())
```

we get:

```
[<class 'main.D'>, <class 'main.B'>, <class 'main.C'>,
<class 'main.Animal'>, <class 'object'>]
```

So the constructor calls proceed like this:

```
D.init()
B.init()
C.init()
Animal.init()
C finishes
B finishes
D finishes
```

The important result is that `Animal.init()` runs exactly once.
This is why we prefer:

```
super().init()
```

instead of directly writing:

```
Animal.init(self)
```

Directly calling the base class constructor can accidentally initialize the same part of the object more than once. Using `super()` allows all classes in the inheritance hierarchy to cooperate.

1.10 What We Will Use in This Course

In CPSC 250, we will primarily use single inheritance.

The pattern you should follow is:

```
class Student(Person):
    def init(self, name, student_id):
        super().init(name)
        self.student_id = student_id
```

Whenever a derived class defines its own constructor, it should generally call the parent constructor using `super()`.

This ensures that inherited data members are properly initialized and that the entire object is constructed correctly.

1.11 Key Takeaways

- A derived class inherits data and behavior from its base class.
- Defining a new constructor replaces the inherited constructor.
- The parent constructor is typically called using `super()`.
- Python chooses explicit constructor chaining rather than automatic constructor chaining.
- Multiple inheritance creates the Diamond Problem.
- Python resolves this ambiguity using the Method Resolution Order (MRO).
- `super()` follows the MRO rather than simply calling a parent class.